Darshan UNIVERSITY
योग: कर्मसु कौशलम्

ML & DL

23010101014 | Vishal Baraiya | 635

Lab - 9

Step 1: Import Libraries

This step imports all necessary libraries for data processing, visualization, and machine learning.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
```

Step 2: Load the Dataset

Load Given dataset - heart.csv

In [2]:
```
df = pd.read_csv("heart.csv")
df.head()
```

Out[2]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	t
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	

Step 3: Data Overview

In this step, we examine the dataset structure, summary statistics, and check for missing values.

In [3]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   age       1025 non-null   int64  
 1   sex       1025 non-null   int64  
 2   cp        1025 non-null   int64  
 3   trestbps  1025 non-null   int64  
 4   chol      1025 non-null   int64  
 5   fbs       1025 non-null   int64  
 6   restecg   1025 non-null   int64  
 7   thalach   1025 non-null   int64  
 8   exang     1025 non-null   int64  
 9   oldpeak   1025 non-null   float64
 10  slope     1025 non-null   int64  
 11  ca        1025 non-null   int64  
 12  thal      1025 non-null   int64  
 13  target    1025 non-null   int64  
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

In [4]:
```
df.describe()
```

Out[4]:

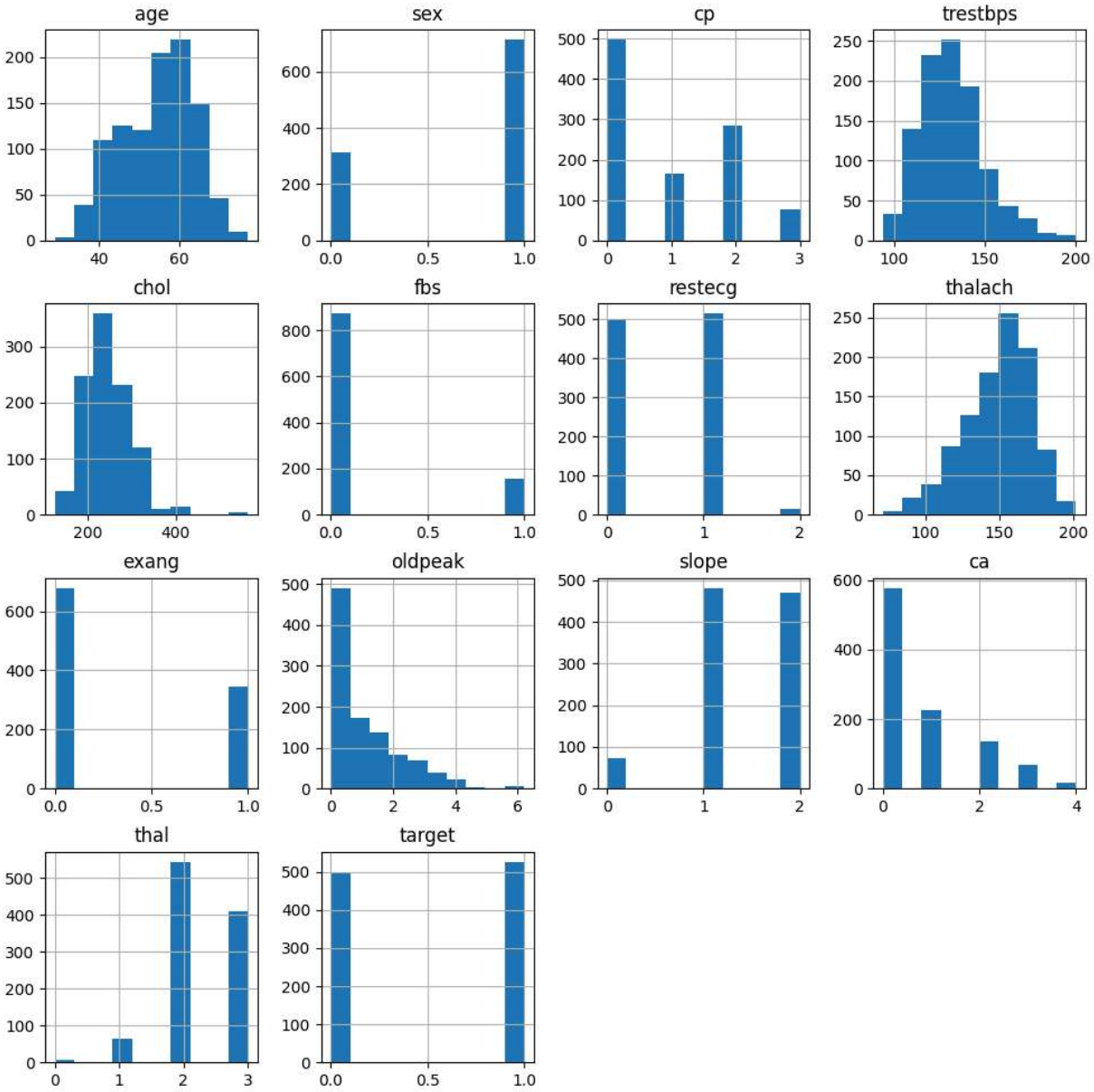
	age	sex	cp	trestbps	chol	fbs	r
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.00000	1025.000000	1025.0
mean	54.434146	0.695610	0.942439	131.611707	246.00000	0.149268	0.5
std	9.072290	0.460373	1.029641	17.516718	51.59251	0.356527	0.5
min	29.000000	0.000000	0.000000	94.000000	126.00000	0.000000	0.0
25%	48.000000	0.000000	0.000000	120.000000	211.00000	0.000000	0.0
50%	56.000000	1.000000	1.000000	130.000000	240.00000	0.000000	1.0
75%	61.000000	1.000000	2.000000	140.000000	275.00000	0.000000	1.0
max	77.000000	1.000000	3.000000	200.000000	564.00000	1.000000	2.0

Step 4: Univariate Analysis

Here we visualize the distribution of each feature using histograms.

In [5]:
```
df.hist(figsize=(12,12))
```

Out[5]:
```
array([[<Axes: title={'center': 'age'}>, <Axes: title={'center': 'sex'}>,
        <Axes: title={'center': 'cp'}>,
        <Axes: title={'center': 'trestbps'}>],
       [<Axes: title={'center': 'chol'}>,
        <Axes: title={'center': 'fbs'}>,
        <Axes: title={'center': 'restecg'}>,
        <Axes: title={'center': 'thalach'}>],
       [<Axes: title={'center': 'exang'}>,
        <Axes: title={'center': 'oldpeak'}>,
        <Axes: title={'center': 'slope'}>,
        <Axes: title={'center': 'ca'}>],
       [<Axes: title={'center': 'thal'}>,
        <Axes: title={'center': 'target'}>, <Axes: >, <Axes: >]],
      dtype=object)
```



Step 5: Bivariate Analysis

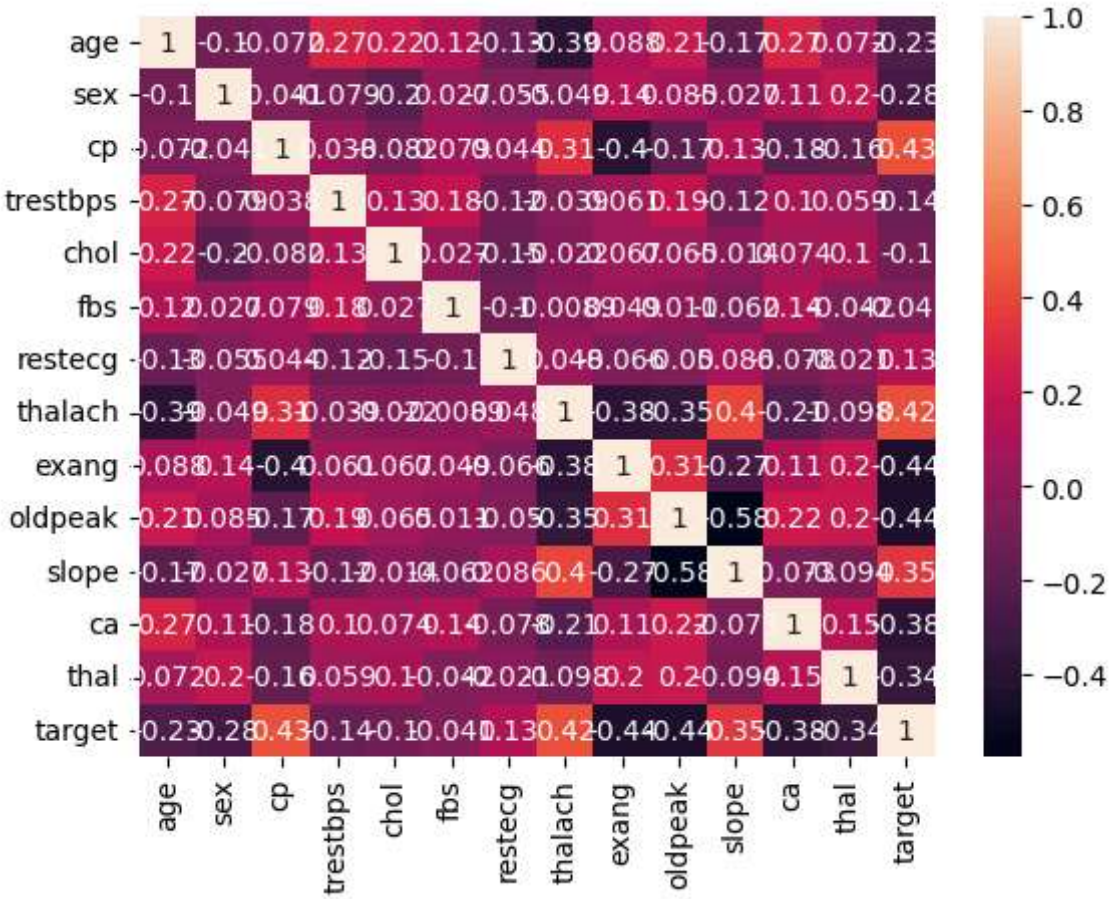
This step involves exploring the correlations between features using a heatmap.

In [6]:
```
import seaborn as sns
```

In [7]:
```
# %matplotlib qt
```

In [8]:
```
sns.heatmap(df.corr(),annot=True)
```

Out[8]: <Axes: >

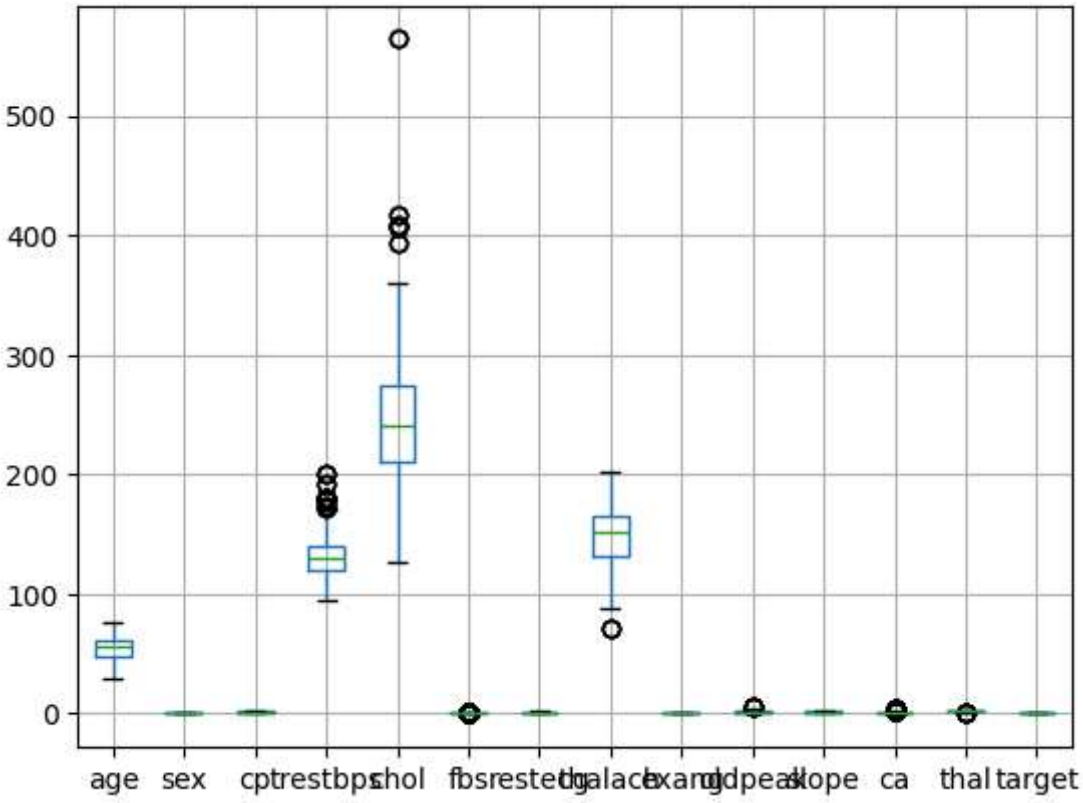


Step 6: Outlier Detection

We visualize potential outliers using boxplots.

In [9]:
```
df.boxplot()
```

Out[9]:
```
<Axes: >
```



Step 7: Split Data into Training and Testing Sets

The dataset is split into training and testing sets for model evaluation.

In [10]:
```
x = df.iloc[::,:-1:]
```

In [11]:
```
y = df['target']
```

In [12]:
```
from sklearn.model_selection import train_test_split
```

In [13]:
```
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.3,random_state=80)
```

Step 8: Train Decision Tree

We train a Decision Tree Classifier on the training data. You have to also check for KNeighborsClassifier and GaussianNB

In [14]:
```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

In [15]:
```
df = DecisionTreeClassifier()
```

In [16]:
```
df.fit(x_train, y_train)
```

Out[16]:
```
▾ DecisionTreeClassifier
▶ Parameters
```

In [17]:
```
y_pred = df.predict(x_test)
```

In [18]:
```
print("accuracy : ", accuracy_score(y_test, y_pred))
```

```
accuracy :  0.9707792207792207
```

In [19]:
```
df.score(x_test, y_pred)
```

Out[19]:
```
1.0
```

Step 11: Train Bagging Classifier

We train a Bagging Classifier with Decision Trees as the base model.

In [45]:
```
from sklearn.ensemble import BaggingClassifier
```

In [21]:
```
bc = BaggingClassifier(estimator=DecisionTreeClassifier(),n_estimators=5)
```

In [22]:
```
bc.fit(x_train,y_train)
```

Out[22]:
```
▸ BaggingClassifier
▸ estimator: DecisionTreeClassifier
  ▸ DecisionTreeClassifier
```

In [23]:
```
y_predict = bc.predict(x_test)
```

Step 12: Evaluate Bagging Classifier

The Bagging model is evaluated using accuracy.

In [24]:
```
accuracy_score(y_test,y_predict)
```

Out[24]:
```
0.9837662337662337
```

Step 13: Train Random Forest

We train a Random Forest Classifier on the dataset.

In [25]:
```
from sklearn.ensemble import RandomForestClassifier
```

In [26]:
```
rfc = RandomForestClassifier(n_estimators=100,max_features='sqrt')
```

In [27]:
```
rfc.fit(x_train,y_train)
```

Out[27]:
RandomForestClassifier

▶ Parameters

In [28]:
```
y_predict = rfc.predict(x_test)
```

Step 14: Feature Importance in Random Forest

We analyze feature importance as determined by the Random Forest model.

In [29]:
```
rfc.feature_importances_
```

Out[29]:
```
array([0.09915455, 0.03093519, 0.13845294, 0.06978083, 0.07842597,
       0.01051857, 0.01964677, 0.11172473, 0.04381303, 0.12678369,
       0.04420242, 0.13782455, 0.08873677])
```

In [30]:
```
rfc.feature_names_in_
```

Out[30]:
```
array(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
       'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal'], dtype=object)
```

Step 15: Evaluate Random Forest

We evaluate the Random Forest model using accuracy.

In [31]:
```
accuracy_score(y_test,y_predict)
```

Out[31]:
```
0.9902597402597403
```

Step 16: Train AdaBoost Classifier

We train an AdaBoost Classifier on the dataset.

In [32]:
```
from sklearn.ensemble import AdaBoostClassifier
```

In [33]:
```
adsc = AdaBoostClassifier(n_estimators=1000)
```

In [34]:
```
adsc.fit(x_train,y_train)
```

Out[34]:
AdaBoostClassifier

▶ Parameters

In [35]:
```
y_predict = adsc.predict(x_test)
```

Step 17: Evaluate AdaBoost Classifier

The AdaBoost model is evaluated using accuracy.

In [36]:
```
accuracy_score(y_test,y_predict)
```

Out[36]:
```
0.9383116883116883
```

<---- B ---->

Cost Complexity Pruning for Decision Tree

In [42]:
```
clf = DecisionTreeClassifier(random_state=42)
path = clf.cost_complexity_pruning_path(x_train, y_train)
ccp_alphas, impurities = path.ccp_alphas, path.impurities
```

In [43]:
```
clfs = []
for ccp_alpha in ccp_alphas:
    clf = DecisionTreeClassifier(random_state=42, ccp_alpha=ccp_alpha)
    clf.fit(x_train, y_train)
    clfs.append(clf)
```

In [44]:
```
clfs = clfs[:-1]
ccp_alphas = ccp_alphas[:-1]
```

In [ ]:
```

```

Random Forest with Iterative Training (OOB Error)

In [40]:
```
rf = RandomForestClassifier(oob_score=True, warm_start=True, random_state=42)

oob_error_rates = []
n_estimators_list = list(range(10, 201, 10))
```

```
for n in n_estimators_list:
    rf.set_params(n_estimators=n)
    rf.fit(x_train, y_train)

    # OOB Error = 1 - OOB Score
    oob_error = 1 - rf.oob_score_
    oob_error_rates.append(oob_error)
```

```
C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\ensemble\_forest.py:611: UserWarning: Some inputs do not have OOB scores. This probably means too few trees were used to compute any reliable OOB estimates.
  warn(
```

In [ ]:

Receiver Operating Characteristic (ROC) Analysis

In [41]:
```
from sklearn.metrics import roc_curve, auc


y_prob = rf.predict_proba(x_test)[:, 1]

# Calculate ROC curve metrics
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

# Plot ROC Curve
```

In [ ]: